

INFORME TÉCNICO GRUPAL



Sistema de Archivos Distribuidos por Bloques

Arquitecturas de Nube y Sistemas Distribuidos

Universidad Pontificia Bolivariana · 2026



Integrantes del Grupo

Sebastián Andrés Cruz Sierra · ID: 000208415

Ali Taha Ramírez

Federico Martínez López

Docente: Álvaro Ospina Sanjuan

Fecha de entrega: Mayo 24 de 2026 · Medellín, Colombia



Python 3.11 · FastAPI · Docker · AWS EC2 · REST

TABLA DE CONTENIDO

1. Objetivo

2. Marco Teórico

2.1 Sistemas de Archivos Distribuidos (DFS)

2.2 Google File System (GFS)

2.3 Hadoop Distributed File System (HDFS)

2.4 FastAPI y Comunicación REST

3. Descripción del Servicio y Problema Abordado

3.1 Operaciones del Sistema

4. Arquitectura del Sistema

4.1 Tipo de Arquitectura

4.2 Componentes Principales

4.3 Infraestructura AWS Desplegada

4.4 Tecnologías Utilizadas

5. Especificación de Protocolos y APIs

5.1 Canales de Comunicación

5.2 API del NameNode

5.3 API del DataNode

6. Algoritmos de Particionamiento, Distribución y Replicación

6.1 Resumen de Algoritmos

6.2 Flujo de Escritura (put)

6.3 Flujo de Lectura (get)

6.4 Manejo de Fallos y Re-replicación

7. Entorno de Ejecución

7.1 Estructura del Repositorio

7.2 Arquitectura Docker

7.3 Ejecución Local con Docker Compose

7.4 Despliegue en AWS Academy

8. Pruebas y Análisis de Resultados

8.1–8.6 Pruebas realizadas y resumen

9. Conclusiones

10. Referencias

1. Objetivo

Diseñar e implementar un Sistema de Archivos Distribuido (DFS) minimalista, orientado a bloques, que permita almacenar y recuperar archivos grandes de forma distribuida entre múltiples nodos en Internet. El sistema aplica **particionamiento en bloques**, distribución entre DataNodes y **replicación con factor mínimo de dos**, garantizando disponibilidad y tolerancia básica a fallos.

La implementación sigue una arquitectura tipo HDFS/GFS con un **NameNode central** de metadatos y **DataNodes** de almacenamiento, exponiéndose a través de una CLI y una API REST implementada en Python con FastAPI, desplegada sobre instancias EC2 en AWS Academy.

2. Marco Teórico

2.1 Sistemas de Archivos Distribuidos (DFS)

Un Sistema de Archivos Distribuido (DFS) permite compartir y acceder de forma concurrente a archivos almacenados en diferentes nodos de una red. A diferencia de un sistema de archivos local, el DFS abstrae la distribución física de los datos, ofreciendo al usuario una vista coherente y unificada independientemente de dónde residan los archivos. Existen dos enfoques principales:

- **DFS basado en bloques:** la unidad de lectura/escritura es el bloque. Los bloques se distribuyen entre múltiples nodos. Ejemplos: NFS, AFS, HDFS, GFS.
- **DFS basado en objetos (Object Storage):** la unidad de distribución es el archivo completo, diseñado para un esquema WORM (Write Once, Read Many). Ejemplos: Amazon S3, Google Cloud Storage.

2.2 Google File System (GFS)

GFS, presentado por Google en 2003, introdujo el modelo de arquitectura maestro-trabajador para DFS a gran escala. Sus principios fundamentales son: un único Master que gestiona los metadatos; múltiples ChunkServers que almacenan datos en **chunks de 64 MB**; replicación automática de chunks; y optimización para cargas de trabajo de lectura secuencial y archivos grandes. El paper fundacional de Ghemawat, Gobioff y Leung (2003) demostró que es posible construir un sistema de alta disponibilidad sobre hardware commodity mediante replicación agresiva y detección proactiva de fallos.

2.3 Hadoop Distributed File System (HDFS)

HDFS es la implementación open-source del modelo GFS, parte del ecosistema Apache Hadoop. Mantiene los mismos principios: NameNode central, DataNodes de almacenamiento, bloques de **128 MB** por defecto y factor de replicación configurable. HDFS introduce el concepto de **heartbeat**: cada DataNode envía señales periódicas al NameNode para indicar que sigue activo. Si el NameNode no recibe heartbeat en un umbral definido, considera el nodo caído e inicia la re-replicación de los bloques perdidos.

2.4 FastAPI y Comunicación REST

FastAPI es un framework web moderno para Python basado en estándares OpenAPI y JSON Schema. Ofrece alto rendimiento gracias a Starlette y Uvicorn, validación automática con Pydantic, documentación interactiva autogenerada (Swagger UI / ReDoc) y soporte nativo para operaciones asíncronas. Su adopción facilita la implementación de los endpoints del NameNode y los DataNodes bajo una arquitectura stateless propia de REST sobre HTTP/1.1.

3. Descripción del Servicio y Problema Abordado

El problema central abordado es la necesidad de almacenar y recuperar archivos grandes de forma fiable y eficiente cuando ningún nodo individual tiene capacidad suficiente, o cuando la disponibilidad no puede depender de un único punto de fallo. El DFS implementado resuelve este problema mediante cuatro mecanismos:

- **Particionamiento:** el archivo se divide en bloques de tamaño configurable (por defecto 64 MB). El último bloque puede ser de tamaño menor.
- **Distribución:** el NameNode asigna DataNodes a cada bloque usando round-robin ponderado por capacidad disponible.
- **Replicación:** cada bloque se almacena en al menos 2 DataNodes distintos, garantizando disponibilidad ante la caída de un nodo.
- **Gestión centralizada de metadatos:** el NameNode mantiene un registro consistente de la ubicación de cada bloque.

El sistema adopta el esquema **WORM** (Write Once, Read Many): no se soporta la actualización parcial de archivos, lo que simplifica el control de concurrencia y la gestión de la replicación, consistente con el diseño de GFS y HDFS.

3.1 Operaciones del Sistema

Comando	Descripción
<code>put <archivo></code>	Carga un archivo al DFS, particionándolo y distribuyendo sus bloques entre DataNodes.
<code>get <archivo></code>	Recupera un archivo reconstruyéndolo a partir de sus bloques distribuidos.
<code>ls [directorio]</code>	Lista archivos y directorios en la ruta indicada.
<code>mkdir <dir></code>	Crea un directorio en el namespace del DFS.
<code>rmdir <dir></code>	Elimina un directorio vacío del namespace.
<code>rm <archivo></code>	Elimina un archivo y notifica a los DataNodes la liberación de bloques.
<code>status</code>	Muestra el estado del clúster: NameNode y cada DataNode (activo/caído).
<code>register</code>	Registra un nuevo usuario con autenticación básica (usuario y contraseña).

4. Arquitectura del Sistema

4.1 Tipo de Arquitectura

El sistema adopta una arquitectura **Maestro-Trabajador (Master-Workers)** con elementos de **Cliente-Servidor (C/S)**. El NameNode actúa como maestro centralizado de metadatos; los DataNodes son trabajadores de almacenamiento; y el Cliente interactúa con ambos a través de APIs REST sobre HTTP/1.1.

4.2 Componentes Principales

Componente	Descripción	Tecnología
Cliente (CLI)	Interfaz de línea de comandos. Se autentica con el NameNode, solicita metadatos y transfiere bloques directamente a los DataNodes.	Python + argparse/click
NameNode	Servidor central de metadatos. Mantiene namespace, tabla de bloques (archivo → bloques → DataNodes), registro de nodos activos y estado de replicación. Desplegado en dfs-namenode (100.53.106.28:8000).	FastAPI + Uvicorn + JSON
DataNodes (×3)	Servidores de almacenamiento. Almacenan bloques localmente, envían heartbeats periódicos al NameNode y replican bloques entre sí. Desplegados en dfs-dn1, dfs-dn2, dfs-dn3 (puerto 8001).	FastAPI + Uvicorn + Docker

4.3 Infraestructura AWS Desplegada

Instancia	Rol	IP Pública	Puerto	Tipo EC2
dfs-namenode	NameNode	100.53.106.28	8000	t3.micro
dfs-dn1	DataNode 1	54.236.36.11	8001	t3.micro
dfs-dn2	DataNode 2	3.92.81.197	8001	t3.micro
dfs-dn3	DataNode 3	3.94.22.113	8001	t3.micro

4.4 Tecnologías Utilizadas

Tecnología	Rol en el sistema	Versión
Python	Lenguaje principal de todos los componentes	3.11+
FastAPI + Uvicorn	Framework API REST para NameNode y DataNodes	Última estable
Docker	Contenedores para cada nodo (un contenedor por componente)	Última estable
Docker Compose	Orquestación local del clúster para desarrollo y pruebas	v2

AWS EC2	Infraestructura IaaS — instancias t3.micro con Ubuntu 22.04, región us-east-1a	Academy
argparse / click	CLI del cliente	stdlib Python
JSON / disco	Persistencia de metadatos en el NameNode	—
HTTP multipart	Transferencia de bloques binarios entre cliente y DataNodes	HTTP/1.1

5. Especificación de Protocolos y APIs

5.1 Canales de Comunicación

Canal	Protocolo	Descripción
Cliente ↔ NameNode	REST / HTTP (FastAPI)	Solicitud de metadatos, autenticación, listado de archivos. Stateless por diseño.
Cliente ↔ DataNode	REST / HTTP multipart	Transferencia directa de bloques binarios. Multipart soporta payloads grandes.
NameNode ↔ DataNode	REST / HTTP (FastAPI)	Heartbeat periódico (cada 5 s), registro de nodos, reporte de bloques.
DataNode ↔ DataNode	REST / HTTP multipart	Replicación de bloques entre pares, iniciada tras confirmación del NameNode.

5.2 API del NameNode

Método	Endpoint	Descripción
POST	/auth/login	Autenticación usuario/contraseña. Retorna token de sesión.
POST	/auth/register	Registro de nuevo usuario con hash de contraseña.
POST	/files/put	El cliente informa nombre y tamaño. Retorna esquema de bloques y DataNodes asignados.
GET	/files/get?filename={name}	Retorna lista de bloques y DataNodes para recuperar el archivo.
GET	/files/ls?path={path}	Lista archivos y directorios en la ruta indicada.
DELETE	/files/rm?filename={name}	Elimina el archivo y notifica a los DataNodes la liberación de bloques.
POST	/files/mkdir	Crea un directorio en el namespace.
DELETE	/files/rmdir	Elimina un directorio vacío.
POST	/datanode/register	Un DataNode se registra al iniciar, informando IP, puerto y capacidad.
POST	/datanode/heartbeat	Heartbeat periódico con reporte de bloques almacenados y capacidad disponible.
GET	/status	Estado del clúster: NameNode activo y lista de DataNodes con su estado.

5.3 API del DataNode

Método	Endpoint	Descripción
POST	/block/upload	Recibe bloque binario (multipart/form-data). Almacena localmente y replica si se indica nodo secundario.
GET	/block/download?block_id=	Retorna el contenido binario del bloque solicitado.
DELETE	/block/delete?block_id=	Elimina un bloque del almacenamiento local.
GET	/block/list	Lista los IDs de bloques almacenados (usado en heartbeat extendido).

6. Algoritmos de Particionamiento, Distribución y Replicación

6.1 Resumen de Algoritmos

Algoritmo	Descripción
Particionamiento	El archivo se divide en bloques de tamaño configurable (por defecto 64 MB). El último bloque puede ser de tamaño menor al máximo configurado.
Distribución round-robin	El NameNode asigna DataNodes a cada bloque usando round-robin ponderado por capacidad disponible, equilibrando la ocupación del clúster.
Replicación (factor 2)	Cada bloque se almacena en al menos 2 DataNodes distintos. El cliente envía al nodo primario y éste replica al secundario.
Heartbeat	Cada DataNode envía un heartbeat al NameNode cada 5 s. Umbral de 3 ciclos consecutivos perdidos (~15 s) para marcar nodo caído.
Re-replicación	Ante caída de un DataNode, el NameNode instruye a un nodo sano con copia del bloque para replicarlo a otro nodo, restaurando el factor mínimo de 2.

6.2 Flujo de Escritura (put)

1. El cliente se autentica en el NameNode (POST /auth/login).
2. El cliente envía POST /files/put con nombre de archivo y tamaño total.
3. El NameNode calcula el número de bloques: $N = \text{ceil}(\text{tamaño} / \text{block_size})$.
4. Para cada bloque, el NameNode selecciona 2 DataNodes disponibles (primario y réplica) usando round-robin ponderado.
5. El NameNode registra los metadatos y retorna al cliente la lista de bloques con sus DataNodes asignados.
6. El cliente divide el archivo y para cada bloque envía los datos al DataNode primario (POST /block/upload) indicando la dirección del nodo réplica.
7. El DataNode primario almacena el bloque y lo replica al DataNode secundario.
8. Ambos DataNodes confirman la recepción al NameNode vía heartbeat.

6.3 Flujo de Lectura (get)

1. El cliente solicita GET /files/get con el nombre del archivo.
2. El NameNode retorna la lista ordenada de bloques con sus DataNodes primarios y réplicas.
3. El cliente descarga cada bloque desde el DataNode primario (GET /block/download). Si el primario no responde, intenta automáticamente con el nodo réplica.
4. El cliente reconstruye el archivo concatenando los bloques en el orden correcto.

6.4 Manejo de Fallos y Re-replicación

Si un DataNode deja de enviar heartbeats al NameNode por más de **15 segundos** (3 ciclos de 5 s), el NameNode lo marca como caído. Para cada bloque afectado, el NameNode instruye a un DataNode sano que posea una copia

para que lo replique a otro nodo disponible, restaurando el factor de replicación mínimo de 2.

El cliente, al intentar descargar desde un nodo caído, recibe un error de conexión y automáticamente intenta el nodo réplica registrado en los metadatos, sin intervención del usuario.

7. Entorno de Ejecución

7.1 Estructura del Repositorio

```
tadb202610_proyecto_dfs/
├── namenode/          # Servidor FastAPI del NameNode
├── datanode/          # Servidor FastAPI de los DataNodes
├── client/            # CLI del cliente (cli.py)
├── aws/
│   ├── setup_namenode.sh  # Instala Docker y levanta el NameNode
│   ├── setup_datanode.sh  # Instala Docker y levanta un DataNode
│   ├── configure_aws.sh   # Genera .env con IPs reales de AWS
│   └── check_deployment.sh # Verifica todos los componentes
├── evidencias/        # Capturas y evidencias de pruebas
├── docker-compose.yml
├── .env.example
├── DEPLOY.md          # Runbook completo de despliegue
└── README.md
```

7.2 Arquitectura Docker

Cada componente del DFS se ejecuta en un contenedor Docker independiente, permitiendo simular un clúster distribuido tanto en una sola máquina de desarrollo (con Docker Compose) como en múltiples instancias EC2 de AWS Academy.

- **namenode:** imagen Python 3.11-slim con FastAPI y Uvicorn. Expone puerto 8000. Persiste metadatos en JSON en disco con respaldo en volumen Docker.
- **datanode:** imagen Python 3.11-slim con FastAPI y Uvicorn. En AWS, cada instancia corre en una EC2 distinta en puerto 8001. El almacenamiento de bloques se monta en volumen Docker persistente.
- **client:** imagen Python 3.11-slim con la CLI. Se ejecuta desde la máquina del usuario apuntando a las IPs públicas de AWS mediante archivo .env.client.

7.3 Ejecución Local con Docker Compose

```
# Configurar y levantar
cp .env.example .env
docker compose up -d --build
docker compose ps

# Modo desarrollo (sin Docker)
python namenode/main.py
python datanode/main.py DN1 8001
python datanode/main.py DN2 8002
python datanode/main.py DN3 8003
```

7.4 Despliegue en AWS Academy

El sistema se desplegó en **4 instancias EC2 t3.micro** con Ubuntu 22.04 en la región us-east-1a, todas en estado "En ejecución" con verificación 3/3. Flujo de despliegue automatizado:

1. Configurar IPs de las EC2 con `configure_aws.sh`, generando archivos `.env` con las IPs reales.
2. Ejecutar `setup_namenode.sh` en `dfs-namenode`: instala Docker y levanta el contenedor NameNode en puerto 8000.

3. Ejecutar `setup_datanode.sh` en cada una de las tres instancias DataNode: instala Docker y levanta el contenedor DataNode en puerto 8001.
4. Verificar el despliegue completo con `check_deployment.sh --env .env.client`.
5. Usar el cliente local apuntando a AWS: `python client/cli.py --env .env.client <comando>`.

8. Pruebas y Análisis de Resultados

8.1 Entorno de Pruebas

Todas las pruebas se ejecutaron sobre el entorno distribuido real en AWS Academy (4 instancias EC2 t3.micro, us-east-1a), con el cliente operando desde una máquina local con Windows (Git Bash). Los archivos se transfirieron a través de Internet usando las IPs públicas de cada instancia.

8.2 Prueba 1 — Registro de Usuario y Estado del Clúster

Se registró un nuevo usuario y se verificó que los tres DataNodes y el NameNode estuvieran activos y respondiendo:

```
$ python client/cli.py --env .env.client register
Nuevo usuario: Ali
Contraseña: 12345
Usuario 'Ali' creado correctamente. Sesión guardada para 'Ali'

$ python client/cli.py --env .env.client status
Estado del cluster DFS:
  NameNode: http://100.53.106.28:8000 [OK]
  DN3: 3.94.22.113:8001 — [OK] activo
  DN2: 3.92.81.197:8001 — [OK] activo
  DN1: 54.236.36.11:8001 — [OK] activo
```

Los 4 nodos respondieron correctamente. El registro de usuario y la sesión autenticada funcionaron sin errores.

8.3 Prueba 2 — Subida y Descarga de Archivo (put / get)

Se subió un archivo PDF de aproximadamente **25.8 MB** y se verificó su correcta subida, listado y recuperación:

```
$ python client/cli.py --env .env.client put prueba1.pdf
Subiendo 'prueba1.pdf' (25884039 bytes)...
Plan recibido: 1 bloque(s) distribuidos en 1 nodo(s)
  Bloque 1/1 -> DN3 (25884039 bytes) [OK]
  Replica -> DN2 [OK]
Archivo 'prueba1.pdf' subido correctamente.

$ python client/cli.py --env .env.client ls
Contenido de '/':
  [FILE] prueba1.pdf (25277.4 KB, 1 bloque(s))

$ python client/cli.py --env .env.client get prueba1.pdf
Descargando 'prueba1.pdf'...
  Bloque 1/1 <- DN3 [OK] (25884039 bytes)
Archivo descargado como 'prueba1.pdf' (25884039 bytes)
```

El archivo (25.8 MB) se subió en 1 bloque, replicado en DN3 (primario) y DN2 (réplica). La descarga fue exitosa desde el nodo primario.

8.4 Prueba 3 — Tolerancia a Fallos (caída de DataNode)

Se simuló la caída del DataNode DN3 (nodo primario del bloque) y se verificó que el sistema continuara operativo mediante la réplica en DN2. Se validó la integridad con hash MD5:

```
$ python client/cli.py --env .env.client status
DN3: 3.94.22.113:8001 — [CAIDO]
```

```

DN2: 3.92.81.197:8001 — [OK] activo
DN1: 54.236.36.11:8001 — [OK] activo

$ python client/cli.py --env .env.client get prueba1.pdf -o recuperado.pdf
Bloque 1/1 <- DN3 [ERROR]: WinError 10061 conexion denegada
(intentando replica...)
Bloque 1/1 <- DN2 [OK] (25884039 bytes)
Archivo descargado como 'recuperado.pdf' (25884039 bytes)

$ md5sum prueba1.pdf recuperado.pdf
2aee89ac492e2749f026e19d5916bc01 prueba1.pdf
2aee89ac492e2749f026e19d5916bc01 recuperado.pdf

```

Resultado clave — Integridad verificada: El hash MD5 idéntico entre el archivo original y el recuperado con failover confirma que la replicación preserva la integridad completa de los datos. El sistema detectó la caída del nodo primario y conmutó automáticamente a la réplica sin intervención del usuario.

8.5 Prueba 4 — Recuperación del Clúster

Tras reiniciar DN3, el clúster recuperó su estado completo de 3 DataNodes activos. DN3 se reregistró automáticamente ante el NameNode al reiniciarse:

```

$ python client/cli.py --env .env.client status
Estado del cluster DFS:
NameNode: http://100.53.106.28:8000 [OK]
DN2: 3.92.81.197:8001 — [OK] activo
DN1: 54.236.36.11:8001 — [OK] activo
DN3: 3.94.22.113:8001 — [OK] activo

```

8.6 Resumen de Resultados

Criterio evaluado	Resultado	Estado
Subida y descarga de archivos sin errores	Archivo de 25.8 MB subido y recuperado correctamente sobre AWS	✓ OK
Replicación en al menos 2 DataNodes	Bloque replicado en DN3 (primario) y DN2 (réplica) automáticamente	✓ OK
Ejecución en al menos 3 nodos distribuidos	4 instancias EC2 en red pública de Internet (us-east-1a)	✓ OK
Tolerancia a fallos: caída de un DataNode	Descarga exitosa desde réplica al caer el nodo primario (DN3)	✓ OK
Integridad del archivo recuperado	MD5 idéntico: 2aee89ac492e2749f026e19d5916bc01	✓ OK
Recuperación automática del nodo caído	DN3 se reregistró al NameNode tras reinicio sin configuración manual	✓ OK

CLI funcional (put, get, ls, status, register)	Todos los comandos operativos sobre el entorno AWS real	✓ OK
Autenticación básica de usuarios	Registro y sesión funcionando correctamente con Basic Auth	✓ OK

9. Conclusiones

- Se implementó exitosamente un DFS minimalista orientado a bloques sobre infraestructura distribuida real en AWS Academy, demostrando la viabilidad de los principios de HDFS/GFS con tecnologías modernas de Python y Docker.
- La arquitectura Maestro-Trabajador con un NameNode central y múltiples DataNodes resultó efectiva para gestionar metadatos y coordinar la distribución y replicación de bloques entre nodos en Internet.
- El mecanismo de heartbeat con umbral de 15 segundos permitió detectar caídas de nodos de forma oportuna, activando automáticamente el failover hacia réplicas disponibles sin intervención del usuario.
- La verificación por MD5 demostró que el sistema garantiza integridad de datos en todos los escenarios probados: subida normal, descarga normal y descarga con failover a nodo réplica.
- El uso de Docker y scripts automatizados de despliegue (aws/) facilitó enormemente la portabilidad entre entorno local y entorno distribuido en AWS, reduciendo la complejidad operativa.
- La adopción del esquema WORM simplificó significativamente el control de concurrencia y la gestión de la replicación, decisión consistente con el diseño de GFS y HDFS, a costa de no soportar actualizaciones parciales de archivos.

10. Referencias

- Ghemawat, S., Gobioff, H., & Leung, S.-T. (2003). *The Google File System*. Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP). ACM.
- Shvachko, K., Kuang, H., Radia, S., & Chansler, R. (2010). *The Hadoop Distributed File System*. 2010 IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST). IEEE.
- FastAPI Documentation. (2024). *FastAPI — A modern, fast web framework for building APIs with Python*. <https://fastapi.tiangolo.com>
- Docker Documentation. (2024). *Docker overview*. <https://docs.docker.com/get-started/overview/>
- Apache Hadoop. (2024). *HDFS Architecture Guide*. <https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>
- Amazon Web Services. (2024). *Amazon EC2 Documentation*. <https://docs.aws.amazon.com/ec2/>